COM 769 | Scalable Advanced Software Systems

Ulster University

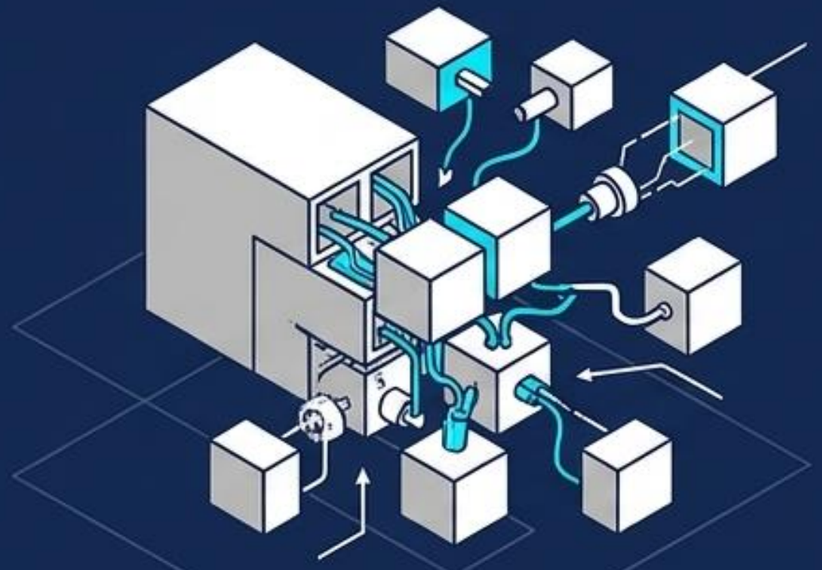
Cloud Design Patterns and Architectures

Frameworks for Strategic Engineering Decisions

Author: Joe Rafferty

The Death of the Monolith

Problem

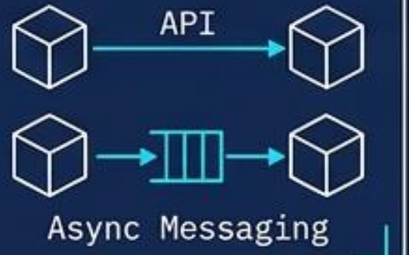


Core Concept: Architecting cloud services fundamentally requires the decomposition of monolithic applications.

Cloud Mechanisms

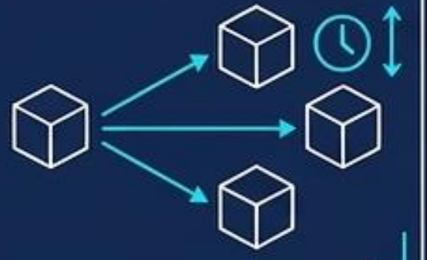
Communication

Services interact via defined APIs or asynchronous messaging/eventing.



Scale

Applications scale horizontally, automatically adding new instances as demand dictates.

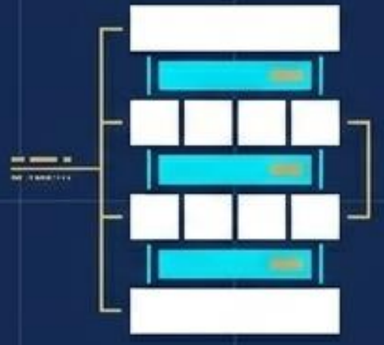


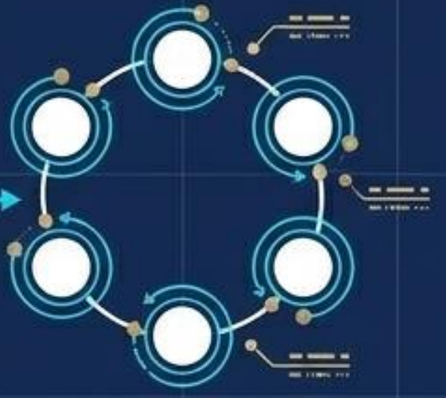
The Architect's First Job: The most critical initial decision is selecting the correct architectural pattern.

Defining Architectural Styles

Definition:

An architecture style is a family of architectures that share specific, identifiable characteristics.



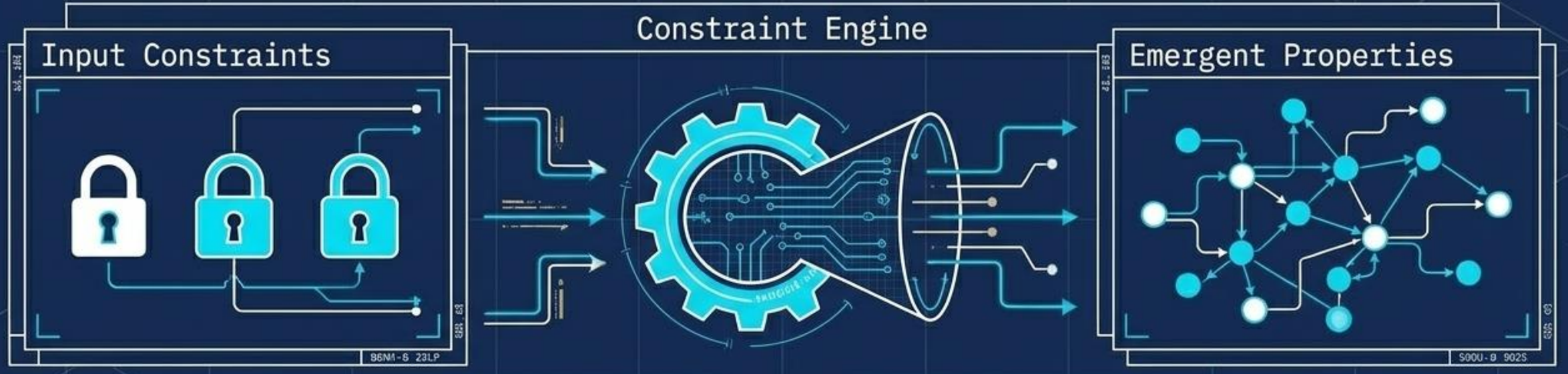


Technology Agnostic:

Architecture styles do not mandate particular technologies. However, certain technologies are natural fits (e.g., containers natively support microservices).

Architecture Styles as Constraints

The Principle: Constraints guide the ‘shape’ of an architecture by actively restricting the range of design choices (e.g., allowed elements and their relationships).



Example: Microservice Constraints

1. A service represents a single responsibility.
2. Every service is completely independent.
3. Data is strictly private to the owning service.

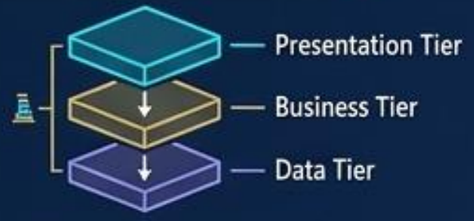
The Emergent Freedom:

Adhering to these strict rules yields massive operational freedom: independent deployments, isolated faults, rapid updates, and easy technology.

Architect’s Note: Be pragmatic. Sometimes relaxing a constraint is better than insisting on rigid architectural purity.

The 7 Primary Cloud Architecture Styles

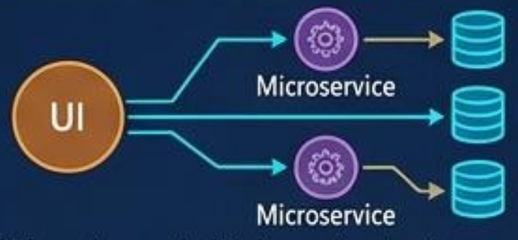
N-Tier



Layered separation.

Layered separation.

Microservices



Small, loosely coupled independent business capabilities.

Independent scaling, deployment.

Event-Driven

Publish-subscribe (pub-sub) model for decoupled producers/consumers.

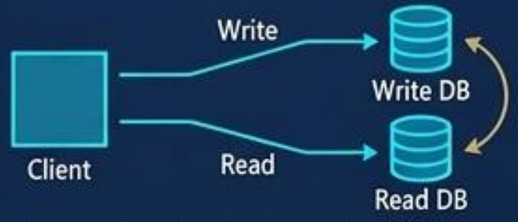
Asynchronous communication.

Web-Queue-Worker

Front-end coupled with backend async processing.

Scalable backend workloads.

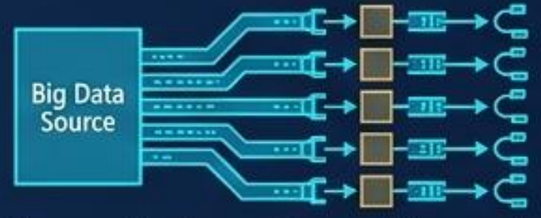
CQRS



Separating read and write operations into separate models.

Optimized performance.

Big Data

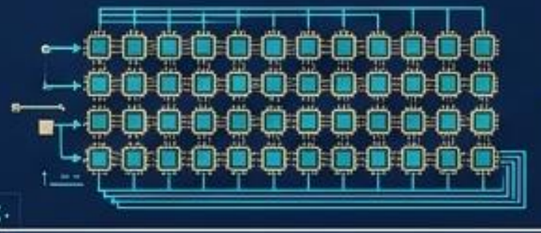


Dividing massive datasets for parallel processing.

Distributed storage & compute.

Big Compute (HPC)

Parallel processing across thousands of cores/specialised GPUs.



High-performance computing, simulations, rendering.

Architectural Comparison Matrix

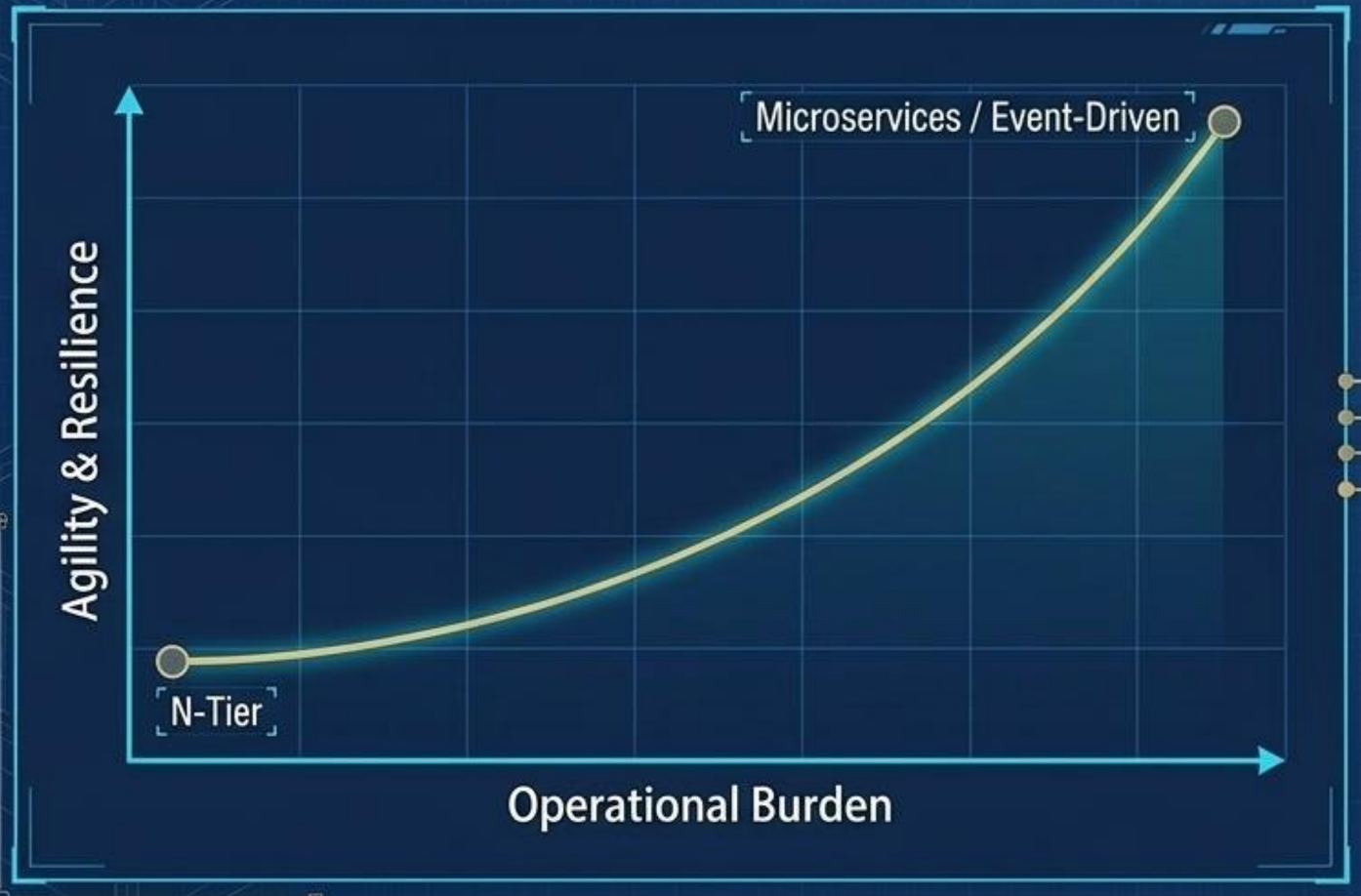
The Cloud Architecture Typology Matrix

Architecture Style	Dependency Management	Domain Type
N-Tier	Horizontal tiers divided by subnet.	Traditional business domains (low update frequency)
Web-Queue-Worker	Front/backend decoupled by async messaging.	Relatively simple domains with intensive tasks
Microservices	Vertically decomposed services via APIs.	Complicated domains (frequent updates)
Event-Driven	Independent views per sub-system.	IoT and real-time systems
Big Data	Divides huge datasets into local chunks.	Batch/real-time analysis & ML
Big Compute	Data allocation to thousands of cores.	Simulation & intensive compute
CQRS	Schema and scale optimized separately.	Collaborative domains with heavy shared data access

Legend: White - Structure, Cyan - Data Flow, Gold - Boundary/Key Concept
Typography: Sans-serif for Headers, Monospaced for Annotations.

The Hidden Cost of Architectural Purity

Core Insight: Every constraint that provides a benefit introduces a corresponding challenge. Ensure the benefits outweigh the burden for your specific bounded context.



Key Trade-offs to Diagnose

Complexity Mismatch

Is the style too complex for the domain? Or too simplistic, risking a "big ball of mud" without clean dependencies?

Asynchronous Messaging

Decouples services and increases reliability, but introduces challenges with eventual consistency and duplicate messages.

Inter-Service Latency

Decomposing systems risks unacceptable network congestion during communication.

Manageability

How difficult will this be to monitor, deploy, and update in practice?

Deep Dive: The N-Tier Blueprint

Definition: N-tier divides an application into logical layers and physical tiers to manage dependencies.

Layers (Logical)

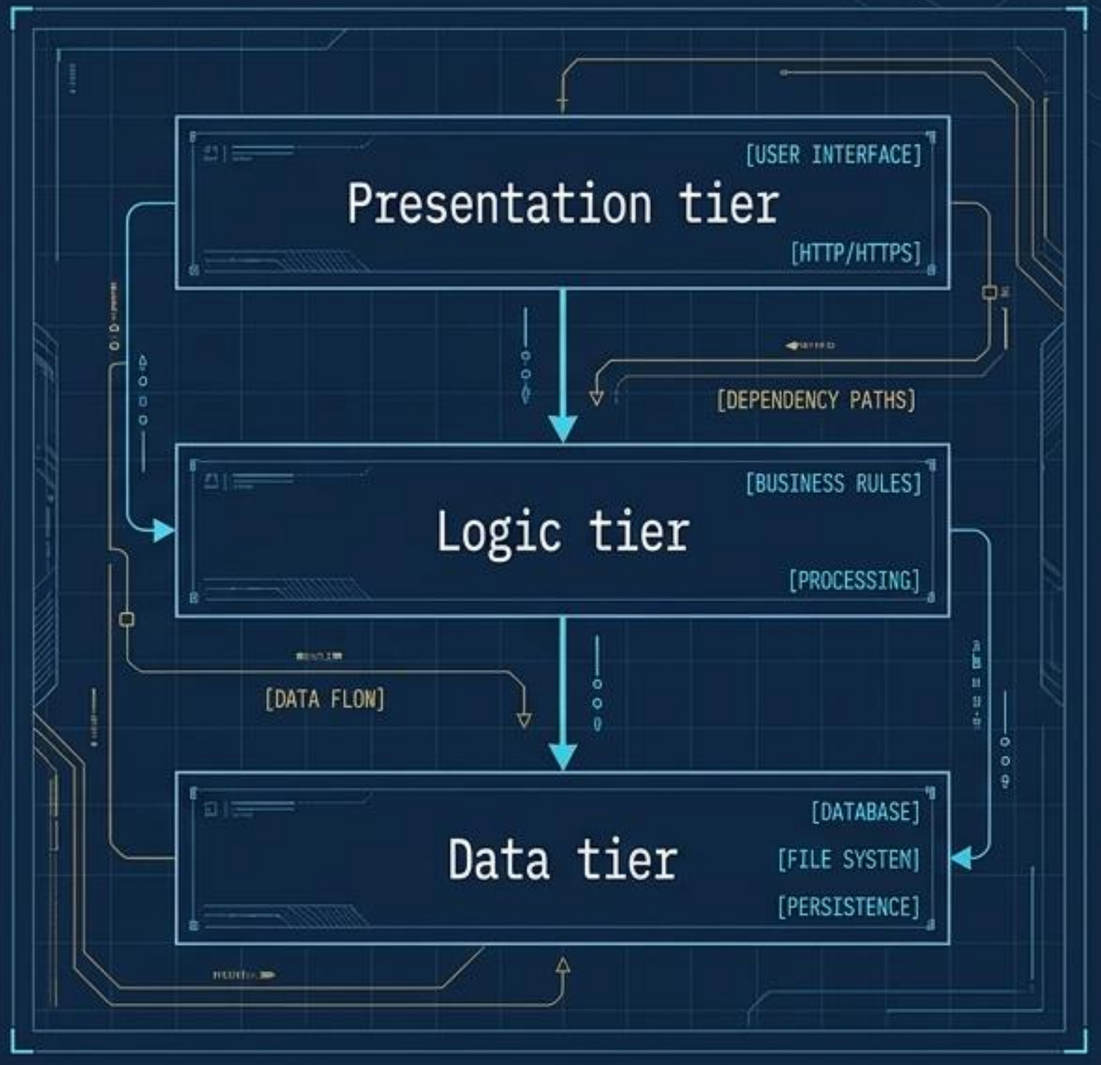
Separate responsibilities (e.g., presentation, business logic, data access). A higher layer uses services in a lower layer, never the reverse.

Tiers (Physical)

Physically separated machines. Tiers improve scalability and resiliency but introduce network latency.

The Middle Tier

A traditional app has Presentation, Middle, and Database tiers. The Middle tier is optional, or can be multiplied to encapsulate different functional areas.



Information Flow: Open vs. Closed Layers

Closed Layer Architecture

A layer can only call the layer immediately below it.

Result: Strictly limits dependencies, but risks generating unnecessary network traffic if a layer merely passes requests along.

Open Layer Architecture

A layer can call any of the layers below it.

Result: More efficient network pathing for direct data retrieval, but creates tighter coupling across the stack.

Evaluating N-Tier: When to Build It

Optimal Use Cases

Simple web applications, migrating on-premises applications to Azure with minimal refactoring, or unified on-prem/cloud development.

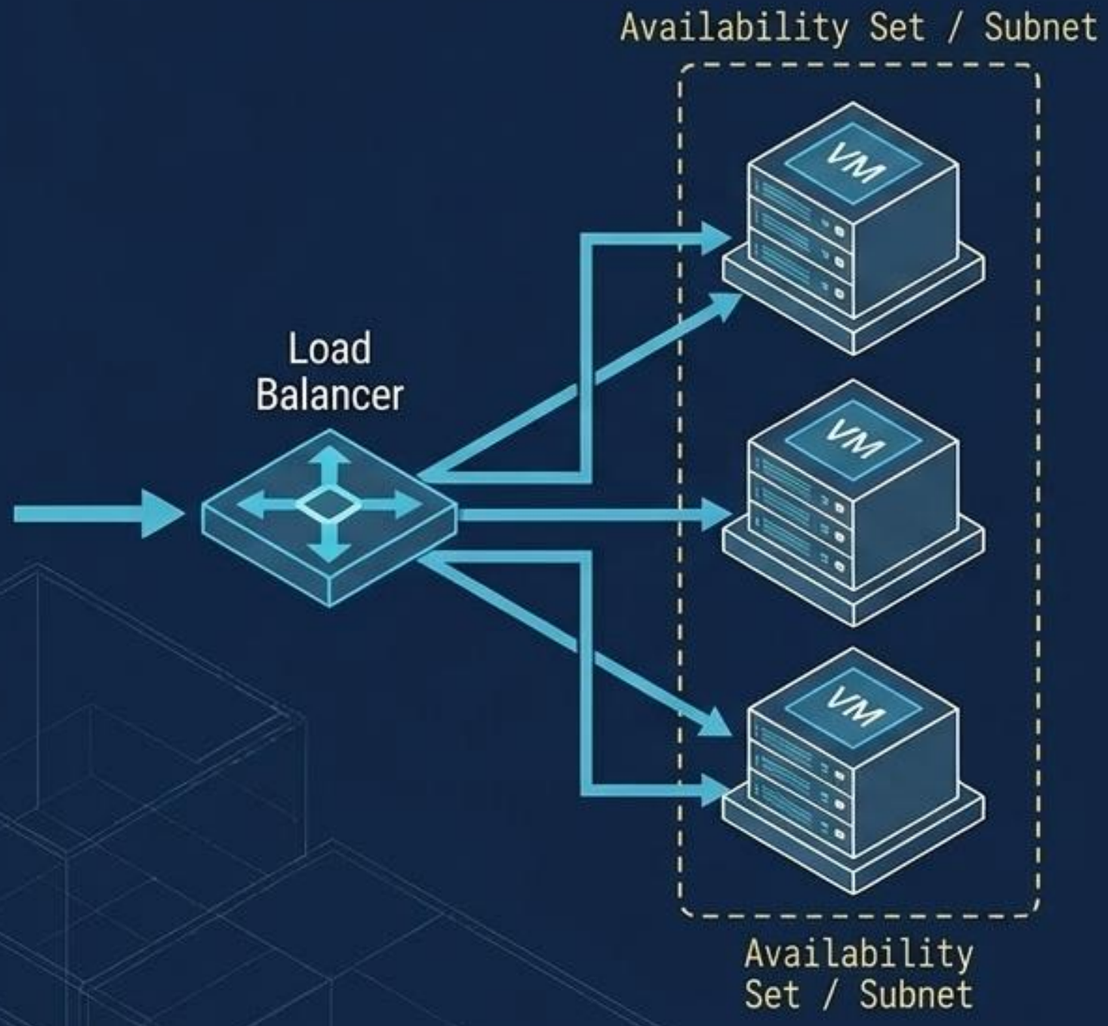
⊕ Benefits (Pros)

- ⊕ High portability between cloud platforms and on-premises environments.
- ⊕ Lower learning curve for developers.
- ⊕ Natural evolutionary step from traditional application models.
- ⊕ Supports heterogeneous environments (Windows/Linux).

⚠ Challenges (Cons)

- ⚠ Middle tiers easily bloat into latency-adding CRUD operations without useful work.
- ⚠ Monolithic design restricts independent feature deployment.
- ⚠ Managing pure IaaS applications requires significant operational overhead.
- ⚠ Managing network security in large N-Tier systems becomes highly complex.

N-Tier at Scale: VM Sets and Subnets



Horizontal Scaling

Each tier consists of two or more VMs placed in an availability set or virtual machine scale set, providing resiliency against hardware failure.

Traffic Distribution

Load balancers distribute external and internal requests evenly across the VMs in a specific tier.

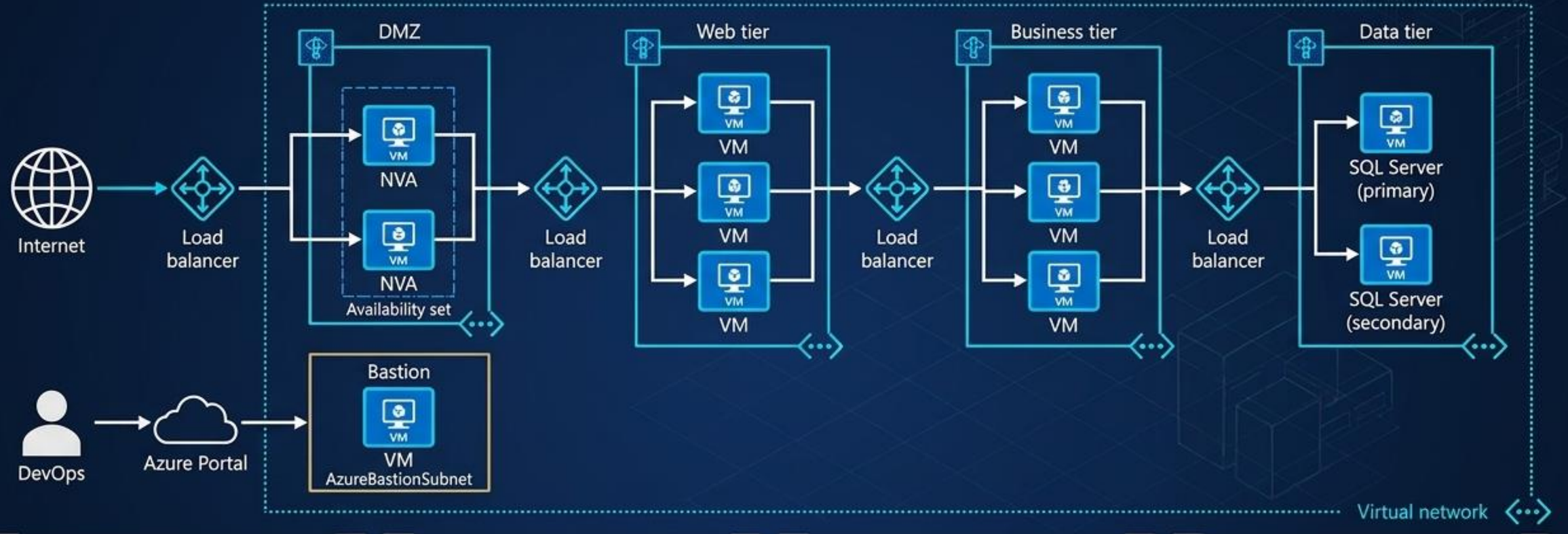
Subnet Boundaries

Placing each tier inside its own dedicated subnet (with internal IP ranges) allows architects to easily apply strict network security group rules and route tables.

State Management

Web and business tiers should remain stateless so any VM can handle any request.

The N-Tier Security Blueprint



The DMZ

Houses Network Virtual Appliances (NVAs) in an availability set for packet inspection and firewalling.

WAF

Placed between the front end and the internet.

Tier Isolation

NSGs restrict access so the Data Tier (SQL Always On) only accepts requests from the Business Tier.

DevOps Bastion

No direct RDP/SSH allowed. Operators connect strictly through a heavily locked-down Bastion host.

Optimization Note: Look for opportunities to inject PaaS (managed services) for caching, messaging, and databases to reduce IaaS overhead.